

1

2

3

4

5

6

7

8
9
10
11
12
13
14
15

16
17
18
19

Input and Output in Hoon

Philip Monk ~wicdev-wisryt
Tlon Corporation

Abstract

Contents

1	Introduction	1
2	The three styles of I/O	2
2.1	Imperative I/O	2
2.2	Monadic I/O	2
2.3	State machines	3
3	A stateful, loopy example	3
4	In Urbit	5
4.1	Jael: a case study	10
5	Recap	12

1 Introduction

Let’s talk about input and output in Urbit. I won’t say much about how events and effects are processed by the runtime; it suffices to note that the runtime is a normal Unix program

20 which 1) listens for various events, 2) passes them to Arvo (Ur-
 21 bit OS), and 3) processes the effects produced by Arvo. Instead,
 22 I'll focus on how I/O works from the perspective of Hoon pro-
 23 grams, like Arvo or apps running on Arvo. But first, let's re-
 24 view I/O paradigms in other languages. ¹

25 2 The three styles of I/O

26 I will call these styles imperative I/O, monadic I/O, and state
 27 machines (note that this classification refers only to the expe-
 28 rience of using them and the program structure they dictate).
 29 An imperative programming language can use any of these and
 30 a functional programming language can use any of them.

31 2.1 Imperative I/O

32 Imperative languages (such as C and Python) perform I/O by
 33 calling a function in the middle of whatever they were doing.
 34 Anywhere, in any statement, you could have I/O.

35 In this example and others to follow, the task will be to read
 36 a filename from the command line, read a URL out of that file,
 37 and fetch the contents at that URL.

```
38 output = fetch(stripWhitespace(readFile(cliInput())))
```

39 For readability, intermediate variables would be a good idea,
 40 but nothing stops you from nesting I/O this way.

41 2.2 Monadic I/O

42 Languages that use explicit monads for I/O (such as Haskell)
 43 perform I/O by running I/O behind a bind operation, which
 44 I'll notate with $\leftarrow$. Thus, while you can't put I/O *anywhere*, the
 45 general structure is a sequence of I/O operations, such as:

```
46 fileName ← cliInput()  
47 url ← readFile(fileName)  
48 output ← fetchUrl(stripWhitespace(url))
```

¹All examples are in pseudocode, since it's useful to understand this before you're comfortable in Hoon. If you want a challenge, try translating them!

49 Simon Peyton Jones’ “Tackling the Awkward Squad” (2010) is
50 the canonical description of monadic I/O, and does a great job
51 of explaining how the Haskell community decided to use this
52 as their primary I/O style.

53 2.3 State machines

54 Languages that use explicit state machines for I/O (such as Elm)
55 perform I/O by accepting events and producing a list of effects.

```
56 event :: Initialize
57       | CLIInput text
58       | FileInput text
59       | HTTPResponse body
60 effect :: CLIInput
61       | ReadFile fileName
62       | FetchUrl url
63       | Print text
64 main(event) {
65   switch event {
66     Initialize → [CLIInput ~]
67     CLIInput fileName → [ReadFile fileName]
68     FileInput url → [FetchUrl stripWhitespace(url)]
69     HTTPResponse body → [Print body]
70   }
71 }
```

72 The effects produced are not functions but data.

73 It’s important to note that this example is chosen to high-
74 light a particular kind of I/O – sequences of events. Further,
75 this example has no persistent state – each I/O operation is a
76 pure function of the output of the previous one.

77 3 A stateful, loopy example

78 Let’s look at another type of program: a simple HTTP server
79 that you can give text at the CLI, which will respond to all HTTP
80 requests with that text.

81 Imperative:

```
82 text = "no text entered yet"
83 while (true)
84   cli = readCli()
85   if (cli != Null)
86     text = cli
87   request = readHttp()
88   if (request != Null)
89     respond(request, text)
```

90 Monadic:

```
91 loop("no text entered yet")
92 where:
93 loop :: text → I/O Null
94 loop = input ← readInput()
95   switch input
96     CLIInput newText → loop(newText)
97     HTTPRequest request →
98       _ ← respond(request, text)
99       loop(text)
```

100 State machine:

```
101 text = "no text entered yet"
102 main(text ,event) {
103   switch event {
104     CLIInput newText → [newText, Null]
105     HTTPRequest request → [text, HTTPResponse text]
106   }
107 }
```

108 Again, note the response in the state machine is data, not a
109 function.

110 I'm not trying to make a point about which type is better
111 in general. There's a pretty strong argument that in the first
112 example, imperative was the best option, followed by monadic,
113 followed by state machine. There's an equally strong argument
114 that the second example is most naturally represented by the
115 state machine.

116 Look at how state is handled in the monadic example. For-
117 mally, bind composes functions, so that every time you see

118 a ← there’s actually a new function (closure/delimited continu-
119 ation) being created and stored for when we receive a response.
120 All variables in scope are stored in that closure, so we can con-
121 veniently access them later without explicitly storing them.
122 However, the state we preserve over time varies a lot. In this
123 example, it varies between storing loop, [loop input], [loop
124 input newText], and [loop input request].

125 By comparison, the state machine does not store interme-
126 diate values, it only stores its explicit state. While this is some-
127 what less flexible, it puts the permanent state front-and-center.

128 I’ll reiterate that all of these are possible to implement in
129 purely functional languages, and further they can all be im-
130 plemented in terms of each other. State machines are trivially
131 implementable – it’s just a function

```
132 [state event] → [new-state (list effect)]
```

133 Monadic I/O can be implemented directly (as in Haskell)
134 or on top of state machines by storing continuations in your
135 state. Imperative I/O is somewhat more difficult but should be
136 possible through algebraic effects.

137 When inventing a new system, you should consider which
138 of these you intend to support based on the sort of code you
139 expect to see. For example, a scripting language will almost
140 always benefit from imperative (or at least monadic) style.

141 4 In Urbit

142 So much for exploring the space of I/O solutions. In terms of
143 deployed code on Earth, the split is about 90% imperative, 5%
144 monadic, and 5% state machine. In Urbit, the split is about 90%
145 state machine and 10% monadic. This is what I intend to ex-
146 plain and defend.

147 Urbit’s OS is called Arvo; it has several “kernel modules”
148 called vanes, and it has userspace applications. Colloquially
149 these are called “apps,” but when speaking about them abstractly,
150 it’s helpful to use their more specific name: “agents”. Arvo, the
151 vanes, and agents are all structured as explicit state machines.

In addition, there are “threads”, which are structured as monadic I/O. Note there’s only a superficial resemblance to Unix threads: they are not executed in parallel and they don’t share memory, or at least not any more than agents do. They’re simply a computation that takes an argument and produces a result, possibly after doing some I/O, structured monadically.

As you can see, all the lowest levels of Arvo are explicit state machines, and only the highest layer supports monadic I/O. To see why this is the case, let’s list advantages and disadvantages to each type. Note that, while I’m speaking specifically of agents and threads in the context of Urbit, these are fundamental properties of monadic and state machine I/O.

Agents are:

- Upgradable. Since the state is explicit, you can upgrade an agent in-place by supplying a new state machine and a function from the old state type to the new state type. This is very similar to a database migration.
- Robust. A state machine must accept any input at any time. If it receives unexpected input, it must have had explicit error handling for that input, since it’s one of the types in the switch statement. Thus, it’s relatively easy to stay in a consistent state.
- Permanent. Since it’s upgradable and robust, there’s no reason to stop its running.
- Hard to write long sequences of I/O. Long sequences of I/O create many possible states, and you must handle any input in any state. This can result in complex, hard-to-follow code with code paths that are rarely if ever taken.

Threads are:

- Not upgradable in-place. Since the state is implicit, you can only cleanly upgrade by (1) killing the thread and restarting it or (2) letting it resolve to a defined “upgradable state” periodically, usually at the top of a “main loop”. If you do (2), then you are in fact structuring it

as a state machine, so the other advantages of threads are compromised.

- **Fragile.** If we don't receive expected input, the default behavior is to be stuck waiting for input. You must remember to add any error-handling features you want, and it's easy to end up in an inconsistent state.
- **Impermanent.** Since a thread can't be upgraded and may get stuck in case of unexpected input, a thread can't be expected to last forever. It must be "rebooted" from time to time.
- **Easy to write long sequences of I/O.** Just...write the I/O one line at a time.

Arvo proper and the vanes are permanent pieces of software and, for this reason, state machines are more natural.

I'll note that other kernels, like the Unix kernel, use imperative I/O. The difference is that Unix isn't permanent in itself – it only lasts until you reboot. Since your permanent state is stored externally, you don't need to be upgradable or permanent. With sufficient discipline you can write robust C code, so it makes sense to use imperative style. Even so, a lot of C code is written in a state machine style; even if it doesn't need to be upgradable, it's worth it for the robustness and concurrency advantages.

However, Arvo is a single-level store – all of its state is permanent. This is very convenient and eliminates many long-tail bugs, but it also means we need to code for permanence, and that's much easier when you structure your code with explicit state.

Further, Arvo and the vanes rarely perform long sequences of I/O. Generally, they do one thing and emit some effects, similar to the HTTP server example above. Sometimes they "pass through" a request. For example, the Eyre vane passes HTTP requests to userspace agents or threads. In this case, there is a sequence of I/O, which in monadic code would be:

```
loop()
where:
```

```

222     loop = request ← receiveRequest() agent = lookupAgent(request)
223     response ← callAgent(agent, request) _ ← sendResponse(request,
224     response) loop()

```

225 This is clear code to read, but it leaves open the question
 226 of how to respond during the callAgent() function. If another
 227 request comes in, can we handle it in parallel? If we need to
 228 upgrade ourselves before the agent responds, can we make sure
 229 to properly handle the response?

230 The naive solution in monadic code is to say that you can
 231 only handle one request at a time, and if an upgrade happens
 232 you just drop outstanding requests. You can resolve both of
 233 these by introducing a sort of main loop, which is just a way
 234 of saying “turn it into a state machine”.

235 In a state machine, the naive solution is:

```

236     state :: (map request agent) main(state, event) { switch event
237     { HTTPRequest request -> agent = lookupAgent(request) [put(state,
238     request, agent), CallAgent agent request] AgentResponse re-
239     quest response -> [del(state, request), HTTPRespond request re-
240     sponse] } }

```

241 Upgrading this is trivial, since the state (map of requests to
 242 outstanding calls to agents) is explicit. It also trivially handles
 243 concurrent requests.

244 I won’t claim this is easier or harder to write than the monadic
 245 version, but it has the properties we need at this level of the
 246 system.

247 Userspace agents similarly are permanent entities that need
 248 to not lose data or get stuck. For this reason they’re structured
 249 as state machines.

250 A helpful comparison is that explicit state machines are
 251 basically Mealy machines, the kind you might have learned
 252 about in an introductory digital design class. This shouldn’t
 253 be surprising; a digital circuit must always be consistent be-
 254 cause it can’t be manually “rebooted”, and their input and out-
 255 put is highly formalized. By contrast, a Rube Goldberg ma-
 256 chine doesn’t have the same constraints; which is why they’re
 257 a lot more fun!

258 It’s notable also that explicit state machines are consider-
 259 ably more declarative than imperative or monadic I/O. Not ev-
 260 eryone thinks that’s a good thing, but anyone who defends

functional programming should see some advantages to that.

However, userspace sometimes needs to perform long sequences of I/O. Let's look at an example where we want to fetch the front page of a site and the first couple comments on each story. HTTP is unreliable, so on failure we retry five times.

Monadic:

```
topStories ← fetch(topStoriesUrl) loop(topStories)
```

where:

```
loop :: topStories -> I/O Null if topStories is Null return Null
comments ← retryLoop(5, head(topStories)) otherComments ←
loop(tail(topStories)) append(comments, otherComments)
retryLoop :: [n story] -> I/O Comments comments ← fetch-
Comments(story) if comments is HTTPError: if n == 0: bail else:
retryLoop(n-1, story)
```

State machine:

```
state :: Initial | FetchingTop | FetchingStory comments stories
retries | Done comments
main(state, event) { switch event { Initialize -> assert state
== Initial [FetchingTop, Fetch topStoriesUrl] HttpResponse re-
sponse -> switch state { FetchingTop -> [FetchingStory Null
stories 5, FetchComments head(stories)] FetchingStory com-
ments stories retries -> if response is HTTPError: if retries == 0:
[Initial, Null] else: [FetchingStory comments stories retries-1,
FetchComments head(stories)] else: comments = append(response, comments)
if tail(stories) is Null: [Done comments, Null] else: [Fetch-
ingStory append(comments, response) tail(stories) 5, FetchCom-
ments head(tail(stories))] } } }
```

With practice, the state machine version of this can be written correctly. However, (1) it's verbose, (2) the control flow jumps all over the place, and (3) in practice it's challenging to get exactly right. This is still a small example, and it gets much worse as the length and complexity of the I/O sequence grows.

The monadic version, on the other hand, flows cleanly down the page. We can factor out generic functionality like "retry this request n times" to make the essence of the computation clear.

Suppose you have to upgrade this. In the state machine, you can see exactly which stage of the computation you're in and write specific code to upgrade cleanly.

Threads, as we've discussed, can't be upgraded in-place. Let's be real though: we're downloading comment sections, not regulating a nuclear reactor. If we get a better version of this script, just kill the old one and start the new one from scratch. What're a couple of extra HTTP requests?

This suggests a general division of workloads: code which needs to be permanent, upgradable, and concurrent should generally be a state machine. Code with long or complex sequences of I/O and definite termination should generally be a thread.

These aren't completely mutually exclusive, but it's surprising how often they are.

4.1 Jael: a case study

When you need to do something permanent and upgradable, but you also need to do long sequences of I/O, the main trick is to factor your problem into both an agent and a thread (or several).

As a case study, consider Jael, one of our vanes. This maintains our PKI state, which it downloads over HTTP from an Ethereum node. Downloading from an Ethereum node is a complex sequence that starts with fetching its most recent block number, then fetching all the blocks we haven't seen and scanning for transactions we care about. We also maintain a map of block hashes to block number, and if we see that a block's parent doesn't have the hash we expect, that means a reorganization has occurred, so we must "rewind" one block at a time until we find where the fork happened, then restart going forward.

You don't have to understand this sequence of I/O; it's sufficient to see that it's complex and must be exactly right. We used to do this in a state machine directly in Jael. This data is, of course, permanent and needs to be accessible to the rest of the system, so Jael must be a state machine.

However, we had many small mistakes in this sequence of I/O, some of which only appeared during unusual events, such as when an HTTP request failed at the same time as a reorganization happened.

336 We're in a bit of a bind, though: Jael has to be a state ma-
337 chine. The solution is to factor out the act of getting updates
338 into a thread while the main PKI state is stored in the state ma-
339 chine in Jael.

340 This thread has a definite purpose: given the most recent
341 block number we already know about, fetch all the PKI trans-
342 actions since then. The function signature is:

```
343       syncEthereum oldBlockNumber -> I/O [newBlockNumber,  
344       newTxns]
```

345 Jael runs this thread every five minutes and, if it succeeds,
346 then we update our block number and PKI state. If the thread
347 fails, gets stuck, or we need to upgrade it, we just kill the old
348 thread and start a new one. Again, a few extra HTTP requests
349 don't matter.

350 From Jael's perspective, it looks something like:

```
351       state :: State block txs outstandingThread main(state,event)  
352       { switch event { Initialize -> [State o Null Null, StartTimer (now  
353       + five minutes)] TimerFired -> effects = If outstandingThread is  
354       Null then Null else [Kill outstandingThread] newId = newThrea-  
355       dId() [State txs newId, append(effects, StartThread newId syncEthereum(block),  
356       StartTimer (now + five minutes))] ThreadFinished Failure ->  
357       [State block txs Null, Null] ThreadFinished Success newBlock-  
358       Number newTxns -> [State newBlockNumber append(txs,newTxns)  
359       Null, Null] } }
```

360 In other words, from Jael's perspective, it's just a single I/O
361 event that encapsulates arbitrarily complex I/O in the thread.
362 A single I/O event is easy enough, and we can maintain all
363 our permanency and robustness guarantees, because we know
364 the thread will have one of three results: success, failure, or
365 it hangs. If we set a timeout, hang becomes failure, so there's
366 only two possible results: success or failure. If we handle both
367 of those correctly, we've handled all the possible I/O errors that
368 could have occurred.

369 What about upgrading? We've already said we can handle
370 the thread dying for arbitrary reasons, so if an upgrade comes
371 in, we just kill it. We can upgrade Jael's state machine easily
372 since its state is very simple.

5 Recap

Let's recap what we've covered:

- There are three styles of I/O: imperative, monadic, and explicit state machines.
- Urbit supports explicit state machines (agents) and monadic I/O (threads).
- Explicit state machines are preferred for permanent, robust, concurrent code.
- Monadic I/O is preferred for short-lived code with complex I/O.
- If a state machine needs to perform complex I/O, it should encapsulate it in a thread so that, from the perspective of the state machine, it's simple I/O.

Hopefully it's fairly clear why Urbit uses explicit state machines in most situations. More than anything else, our goal is to build software that can last forever. Explicit state machines are a crucial tool for that. ☞